

UI Polish & Component Refinement Lab (Practical)

Welcome to the UI Polish & Component Refinement! In this session, you will polish your React user interface, add HTML5 input validation attributes, introduce a reset state feature, and organize your styles to ensure your frontend is robust and ready for FastAPI backend integration.

1. Concept Overview

In this lab, you will:

- Implement HTML5 client-side validation (`min` , `max` , `step`) to catch invalid inputs before they reach the network.
- Add a reset feature to clear form state and previous predictions easily.
- Clean up your application layout and global styles to give your ML dashboard a professional look.

2. Step-by-Step Lab Instructions

Step 1: Update Global Styles in `src/index.css`

Open `src/index.css` (or `src/App.css`) and add clean global typography and background defaults so your dashboard looks centered and polished:

```
/* src/index.css */
body {
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
  Helvetica, Arial, sans-serif;
  background-color: #f1f5f9;
  color: #1e293b;
  box-sizing: border-box;
}
```

```

*, *:before, *:after {
  box-sizing: inherit;
}

```

Step 2: Refine PredictionForm.jsx with Validation and Reset

Open `src/components/PredictionForm.jsx` and update it to include validation constraints and a reset button:

```

// src/components/PredictionForm.jsx
import { useState } from "react";
import axios from "axios";
import "./PredictionForm.css";

function PredictionForm() {
  const [feature1, setFeature1] = useState("");
  const [feature2, setFeature2] = useState("");
  const [predictionResult, setPredictionResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError(null);
    setPredictionResult(null);

    try {
      // Mock API call (Ready to be replaced with FastAPI endpoint:
      // http://127.0.0.1:8000/predict)
      const response = await
        axios.post("https://jsonplaceholder.typicode.com/posts", {
          feature1: Number(feature1),
          feature2: Number(feature2),
        });

      const simulatedPrediction = (Number(feature1) + Number(feature2)) *
        1.5;

      setPredictionResult({
        prediction: simulatedPrediction.toFixed(2),
        status: "Success",
      });
    } catch (err) {
      setError("Failed to connect to the backend server. Please verify
        your connection.");
      console.error("API Error:", err);
    } finally {
      setLoading(false);
    }
  }
}

```

```

};

const handleReset = () => {
  setFeature1("");
  setFeature2("");
  setPredictionResult(null);
  setError(null);
};

return (
  <main className="form-container">
    <section className="card">
      <h2>ML Model Feature Input</h2>
      <form onSubmit={handleSubmit}>
        <div className="input-group">
          <label htmlFor="feature1">Primary Feature (e.g., Sq Ft /
Hours):</label>
          <input
            type="number"
            id="feature1"
            value={feature1}
            onChange={(e) => setFeature1(e.target.value)}
            placeholder="Enter value (e.g., 1200)..."
            min="0"
            step="any"
            required
          />
        </div>

        <div className="input-group">
          <label htmlFor="feature2">Secondary Feature (e.g., Rooms /
Age):</label>
          <input
            type="number"
            id="feature2"
            value={feature2}
            onChange={(e) => setFeature2(e.target.value)}
            placeholder="Enter value (e.g., 3)..."
            min="0"
            step="any"
            required
          />
        </div>

        <div className="button-group">
          <button type="submit" className="submit-btn" disabled=
{loading}>
            {loading ? "Processing..." : "Run Prediction"}
          </button>
          <button type="button" className="reset-btn" onClick=
{handleReset}>
            Reset

```

```

        </button>
      </div>
    </form>

    {error && <p className="error-message">{error}</p>}}

    {predictionResult && (
      <div className="result-box">
        <h3>Prediction Result:</h3>
        <p>Estimated Output: <strong>{predictionResult.prediction}
      </strong></p>
        <p className="status-success">Status: Ready for FastAPI
Backend</p>
      </div>
    )}
  </section>
</main>
);
}

export default PredictionForm;

```

Step 3: Update PredictionForm.css for Button Layouts

Open `src/components/PredictionForm.css` and add rules for the new button group and reset style:

```

/* Add to src/components/PredictionForm.css */
.button-group {
  display: flex;
  gap: 0.75rem;
}

.submit-btn {
  flex: 2;
  background-color: #3b82f6;
  color: white;
  border: none;
  padding: 0.75rem;
  font-size: 1rem;
  font-weight: 600;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.submit-btn:hover:not(:disabled) {
  background-color: #2563eb;
}

.reset-btn {

```

```
flex: 1;
background-color: #e2e8f0;
color: #475569;
border: none;
padding: 0.75rem;
font-size: 1rem;
font-weight: 600;
border-radius: 6px;
cursor: pointer;
transition: background-color 0.2s;
}

.reset-btn:hover {
  background-color: #cbd5e1;
}
```

Step 4: Verify Module 1 Completion

1. Ensure your Vite development server is running (`npm run dev`).
2. Test input constraints: try submitting empty forms or negative numbers to watch browser validation in action.
3. Click **Run Prediction** and then **Reset** to confirm state management works seamlessly.

Congratulations! You have completed **Module 1 (React UI)**. You now have a fully functional, styled React frontend capable of capturing inputs and managing asynchronous states.

3. Reference Links & Further Reading

- **HTML Client-Side Form Validation:** Guide on utilizing built-in HTML attributes for data integrity.
 - **React State Management Best Practices:** Deep dive into lifting state and handling clean component resets.
-